

Avvale

Configuración y despliegue del Pipeline Monitor

© Avvale
Do not duplicate without written permission

Contenido

Versiones del documento **2**

Introducción **3**

Requisitos **4**

Guía **5**

Configuración preliminar 5

Generación de service keys en SAP BTP 6

Generación de service key para acceder a BTP 6

Generación de una service key para enviar notificaciones 9

Configuración un endpoint para manejar alertas 10

Configuración de variables de entorno 13

Despliegue al entorno Cloud Foundry 14

Versiones del documento

Versión	Fecha	Autor	Comentario
1.0.0	29/10/2026	Manuel Almagro Rivas	Versión preliminar
1.0.1	30/10/2026	Manuel Almagro Rivas	Binding de un host, nueva variable a informar

Introducción

La monitorización de integraciones asíncronas (mediadas por colas JMS) a través de SAP Integration Suite supone un reto operativo para el CCI. Al contrario que las integraciones síncronas, una excepción en una cola intermedia no se propaga hacia el sender, y nadie (salvo los operarios del middleware) tienen conocimiento del suceso.

Esto puede verse complicado por factores adicionales, como un desarrollo complejo (véase el pipeline estándar de SAP), los mecanismos de retry (que pueden generar falsos negativos en el monitor) o la existencia de colas FIFO (*first in, first out*) que pueden atascarse sin que los mensajes se puedan depositar en una DLQ (*dead-letter queue*).

Con esta motivación se ha desarrollado una aplicación cuya utilidad es monitorizar las colas y emitir una alerta que resulte útil al equipo de monitorización. La aplicación está escrita en Node.js y puede correr sobre un espacio Cloud Foundry ajeno al del cliente que se pretende monitorizar. Siendo el eje de facturación la memoria reservada total, se ha pretendido reducir a un mínimo la huella de la aplicación: no dispone de interfaz gráfica de usuario, ni de base de datos anexa. Toda la información que requiere (autenticación, endpoints, parámetros de configuración, etc.) se mantiene como variables del sistema.

Este documento está pensado para que los posibles compañeros que deban configurar esto en el futuro, sin experiencia en procedimientos similares, tengan una guía que no asuma demasiados conocimientos previos.

Si se desea conocer cómo funciona la aplicación, la especificación (SPEC.md) puede encontrarse en el repositorio de GitHub.

Requisitos

Para implementar los pasos de la presente guía es necesario cumplir los siguientes requisitos previos:

De tipo local:

- Un ordenador con Windows/macOS y acceso a internet.
- Tener instalado el cliente de línea de comandos de Cloud Foundry. Enlace.
- Disponer de `git` en línea de comandos.
    - En Windows, abrir una consola (Windows + R, cmd.exe) y escribir el comando "`git -v`". Si nos aparece la versión, está instalado. Si no, hay que instalarlo. Enlace.
    - En macOS está instalado de serie normalmente. Si no lo está, puede instalarse de diversas formas. Enlace.

En SAP BTP:

- Es necesario que el usuario con el que accedemos a la global account de BTP de nuestro cliente pertenezca al espacio CloudFoundry como Developer y/o Manager (enlace) y que sea miembro de la organización como Manager (enlace). El objetivo es ser capaz de crear de manera autónoma dos service keys, sin gasto para el cliente.
- Es necesario lo mismo para el espacio de BTP en el que correrá la aplicación, que puede ser el del cliente (en cuyo caso hemos terminado) o uno de Avvale destinado para pruebas (en cuyo caso hay que repetir los pasos del punto anterior).

Guía

Configuración preliminar

- Empezaremos por clonar el repositorio de la aplicación en nuestro ordenador:

```
git clone https://github.com/manuel-almagro-avvale/PipelineMonitoring

cd PipelineMonitoring
```

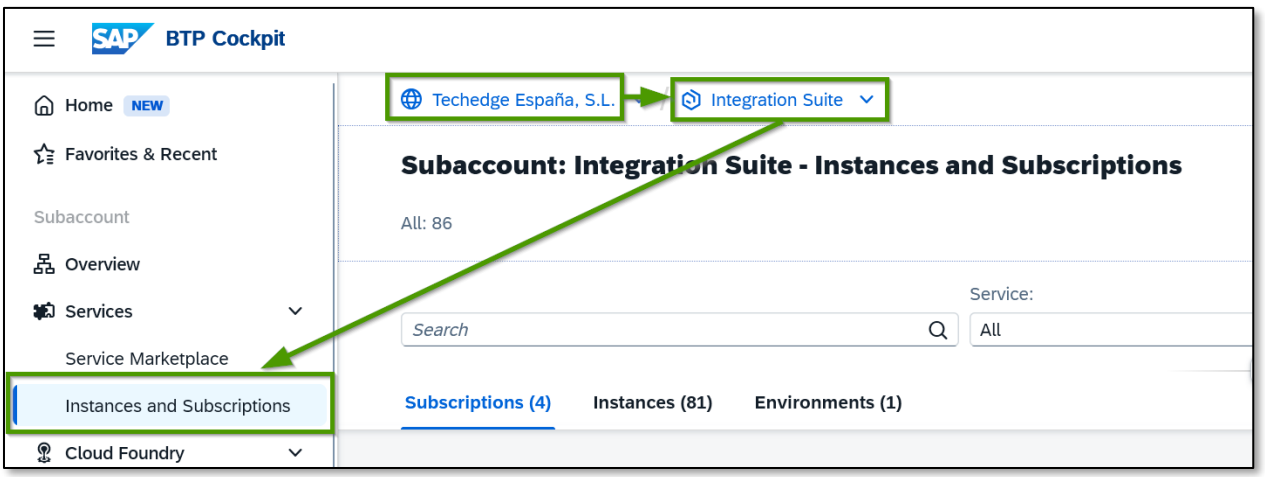
- Ahora, con el repo copiado localmente, tenemos que empezar a configurarlo. Hay un archivo de ejemplo, `vars.example.yml`. Este archivo no puede editarse directamente, sino que ha de crearse una copia y renombrarla a `vars.yml`. El motivo es que aquí introduciremos información sensible, como autenticaciones y endpoints, y el repositorio está preparado para que la información real que vive en este archivo nunca llegue a estar legible en ningún sitio delicado (GitHub, o el propio Cloud Foundry). Esto se configura respectivamente con los archivos `.gitignore` y `.cfignore` que existen en el repo. El renombrado puede hacerse a mano, o dependiendo de tu sistema:
    - En macOS: `cp vars.example.yml vars.yml`
    - En Windows: `copy vars.example.yml vars.yml`
- Con el fichero de configuración vars.yml creado, empezaremos a generar los objetos que necesitamos para poder monitorizar el espacio del cliente.

Generación de service keys en SAP BTP

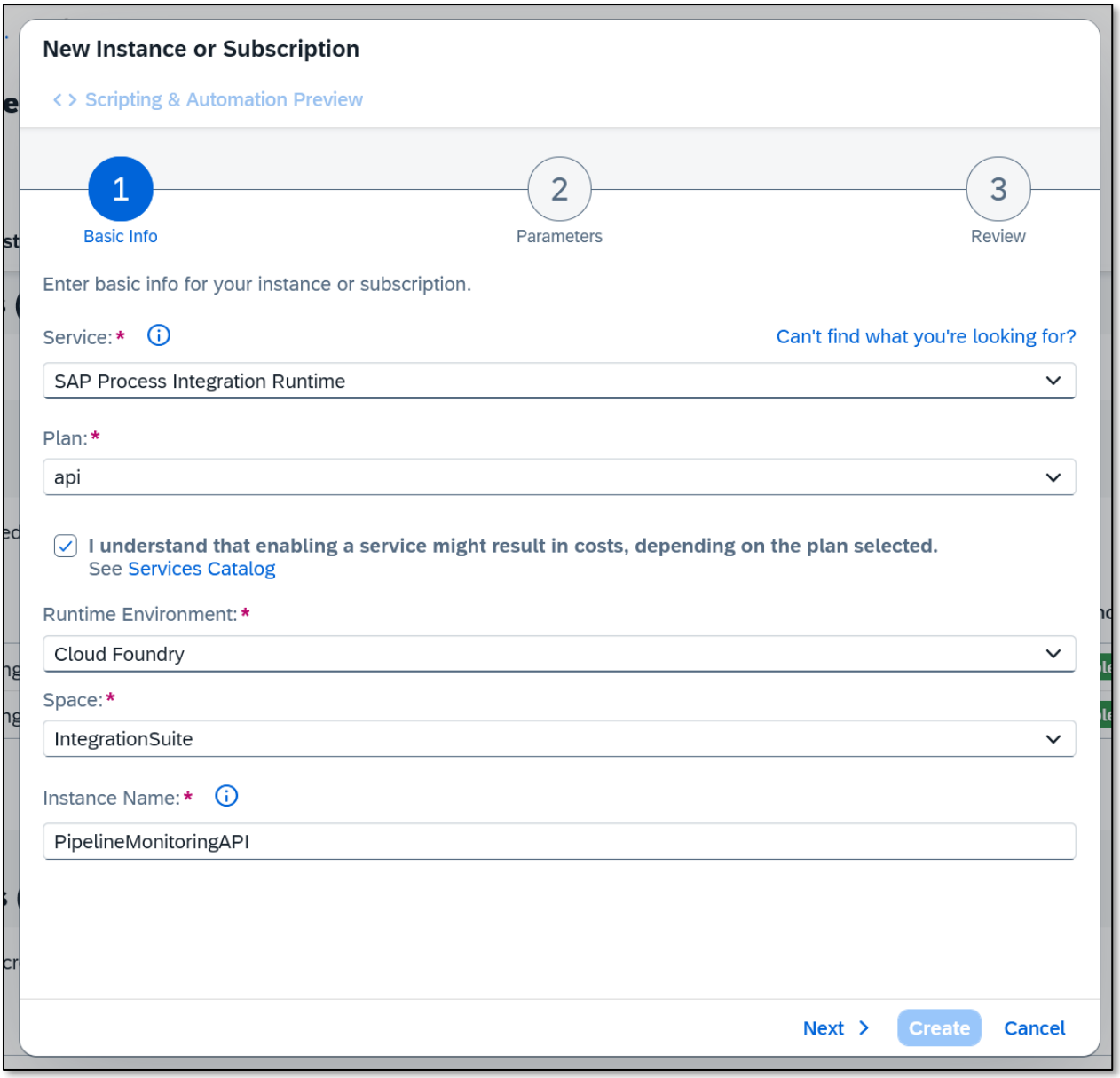
Necesitamos generar dos service keys. Cada service key contendrá las credenciales que usaremos para consumir dos servicios diferentes. La primera nos habilitará el acceso a la API de producto de Integration Suite, y la usaremos para consultar el estado de las colas JMS del cliente. La segunda, nos servirá para que la aplicación pueda enviar un reporte (en forma de JSON) a algún endpoint, donde nosotros podremos manipular esa información de la manera que consideremos oportuna (p.e. generar un mail, abrir un ticket en alguna plataforma, depositar un fichero, etc.).

Generación de service key para acceder a BTP

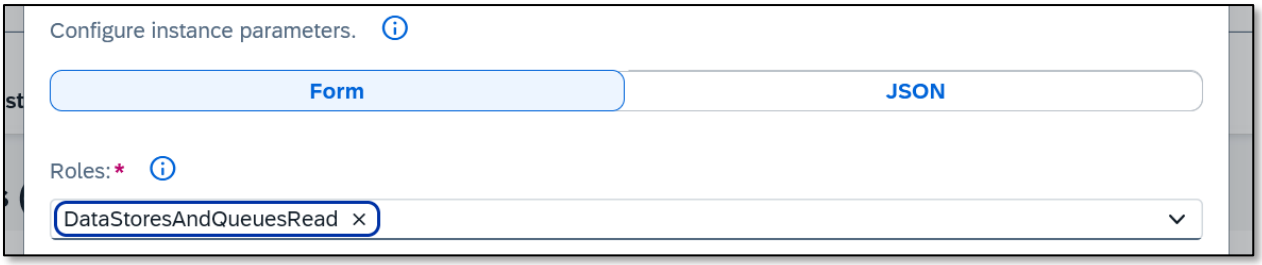
- A través del BTP Cockpit accederemos a la global account del cliente, y a la subaccount del entorno de Integration Suite que vayamos a monitorizar.



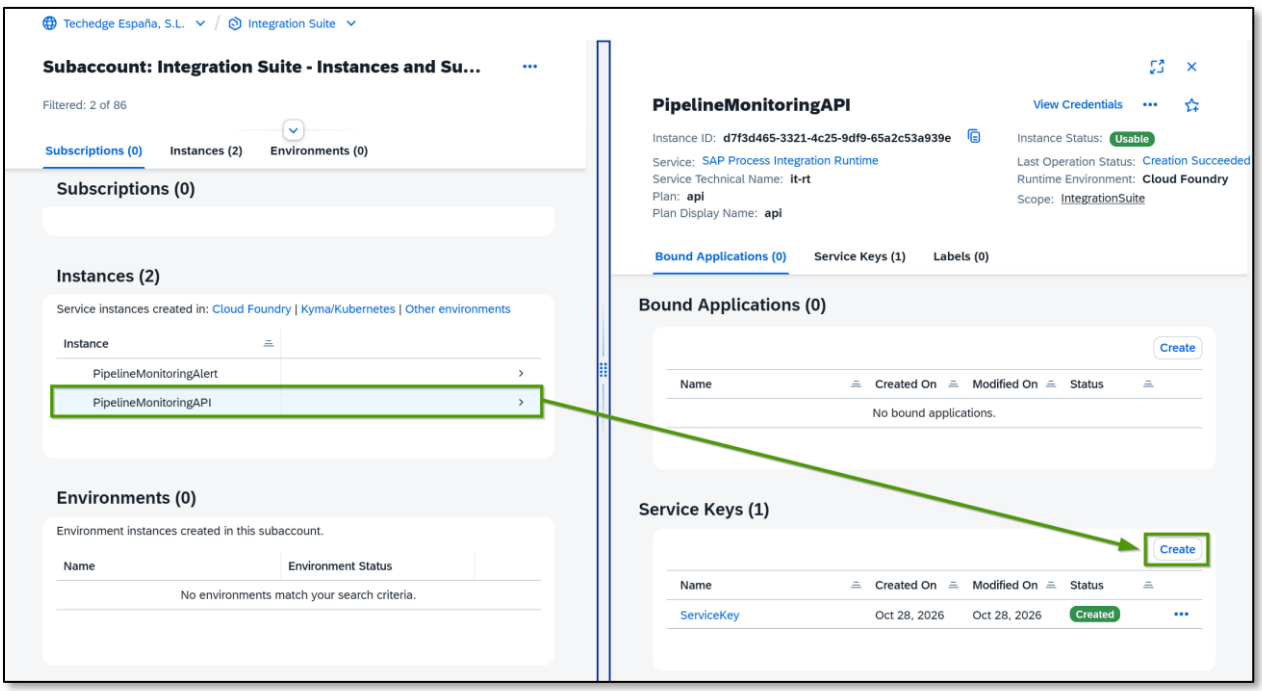
- Haremos clic en Create (arriba a la derecha) para crear una service instance. (La instance es el contenedor de la clave, y es donde se asignan los roles de ésta). Generaremos una instancia configurada como en la captura (el nombre es el único parámetro a nuestra elección, y es recomendable respetar el patrón que haya establecido el cliente en otras instancias). El servicio sobre el que corra deberá ser SAP Process Integration Runtime, y el plan deberá ser api. En general, encontraremos un solo Runtime Environment y un único espacio (que dejaremos por defecto). Si hay varios, deberán coincidir con el runtime y el espacio sobre los cuales esté desplegado Integration Suite.



- En la siguiente pantalla nos pedirán los roles asociados a esta instancia. Añadiremos únicamente el rol DataStoresAndQueuesRead. De esta manera nuestra aplicación sólo será capaz de leer colas, sin modificar nunca nada en el entorno productivo del cliente.



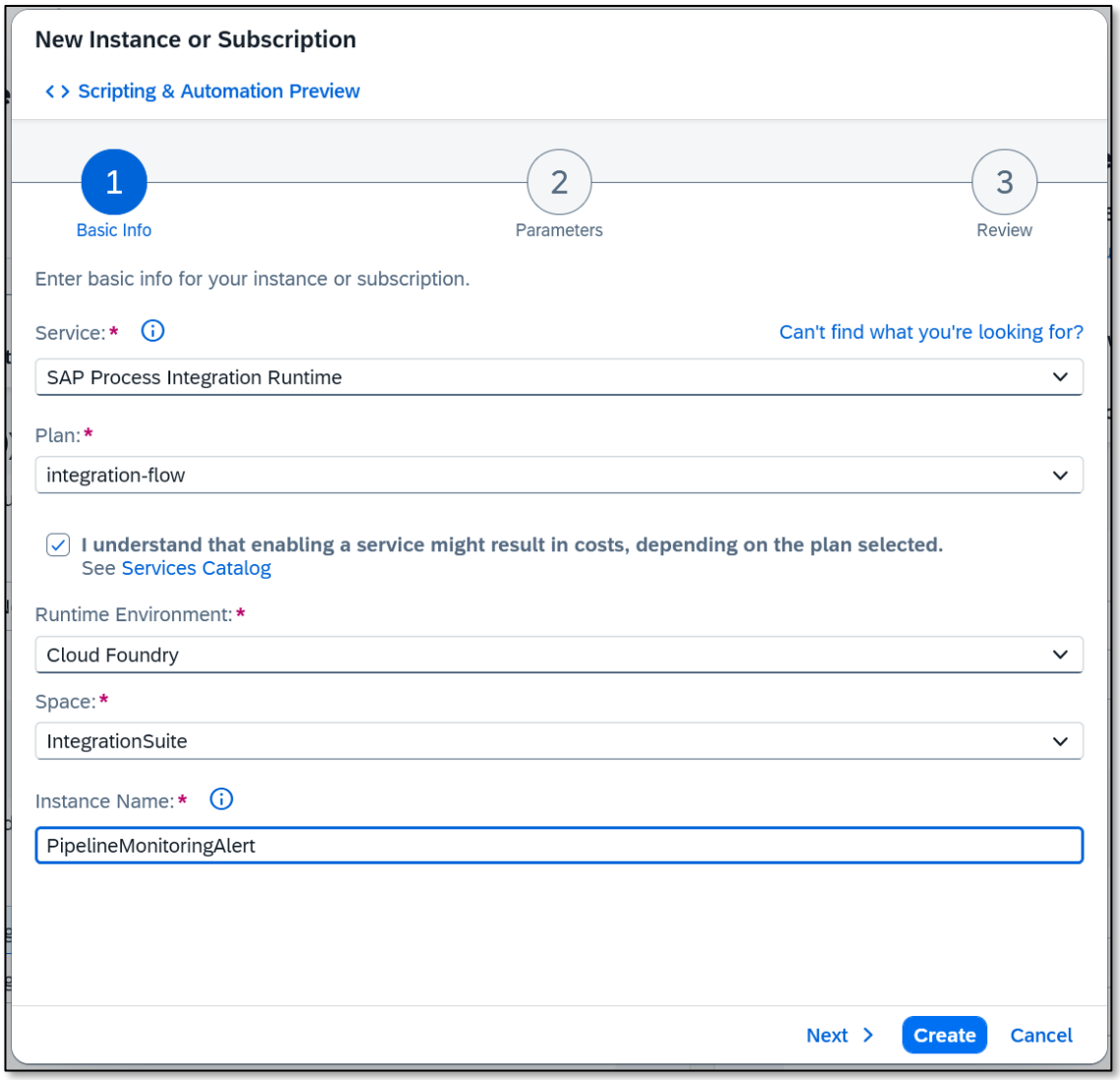
- Haremos clic en Create y esperaremos a que se cree la instancia.
- Con la instancia creada, la abriremos y haremos clic en ella, y bajo el apartado Service Keys haremos clic en Create y mantendremos todos los valores por defecto del menú. El nombre que le demos a la service key es irrelevante, así que es recomendable seguir los patrones establecidos por el cliente en sus otras instancias.



- La service key está creada, y al abrirla veremos un JSON que usaremos después.

Generación de una service key para enviar notificaciones

El proceso es idéntico al paso anterior, con la excepción de que la service instance debe configurarse con el plan integration-flow. Por lo demás, se puede mantener el rol estándar que veremos en la siguiente pantalla (ESBMessaging.send), o configurar uno propio si queremos hacer segregación por roles. Esto es irrelevante para este procedimiento, y depende de lo que elija el desarrollador que maneje la alerta en el middleware más adelante.



Cuando creemos la instancia, crearemos también la service key correspondiente. La usaremos en un paso posterior.

Configuración un endpoint para manejar alertas

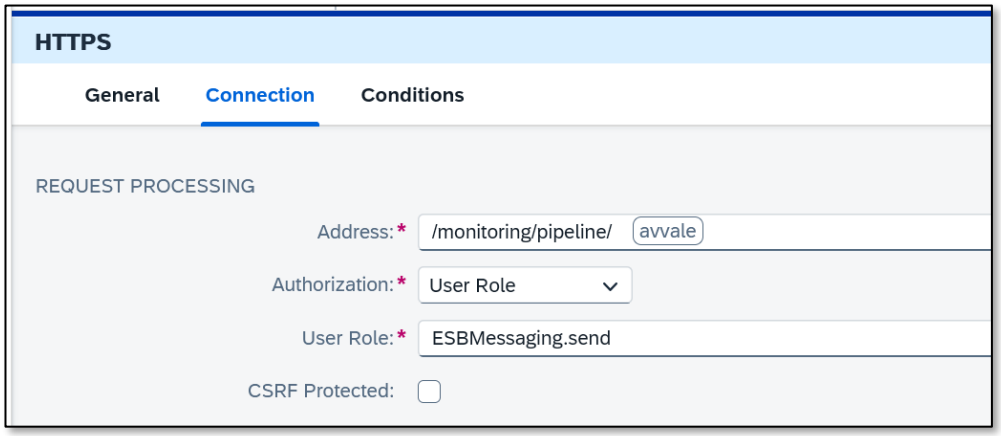
Lo último que tenemos que configurar es un endpoint de integración que reciba alertas y haga con ellas lo que sea necesario. Las llamadas a este endpoint se harán con la service key configurada en el apartado anterior (plan: integration-flow).

Este es el punto más libre de la configuración, porque lo que se hace con la alerta dependerá de cómo se monitoriza cada cliente. El desarrollador de la integración ha de conocer las siguientes directrices:

- En el JSON aparecerán el nombre del cliente, el entorno, y las colas afectadas:

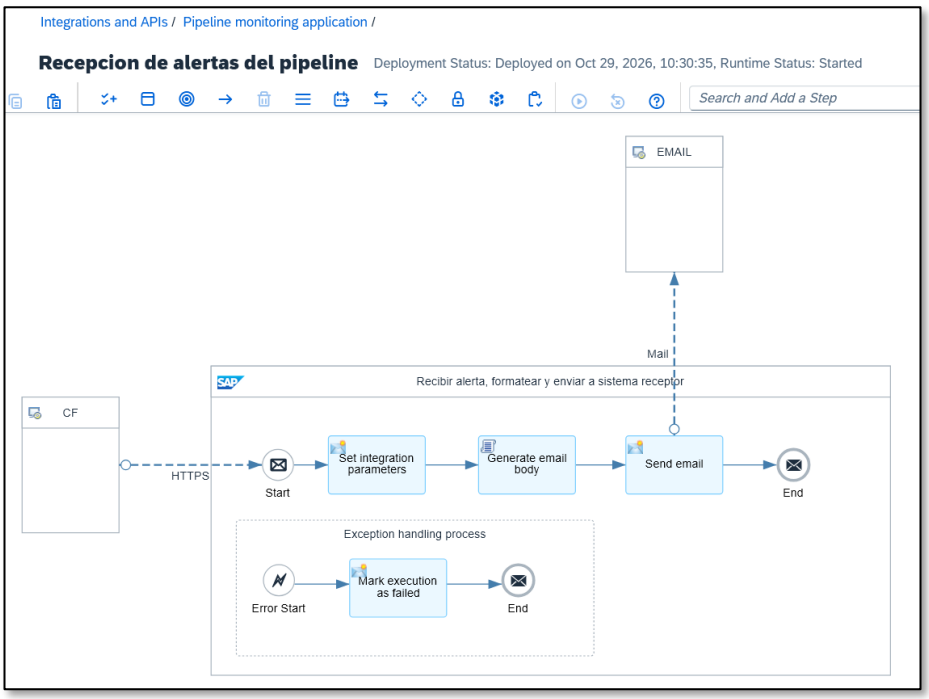
```
{
  "Client": "Avvale",
  "Environment": "dev",
  "Timestamp": "2026-10-29T09:33:20.692Z",
  "AffectedQueues": [
    {
      "QueueName": "DemoQueue",
      "PreviousQueuedMessages": 3,
      "QueuedMessages": 3,
      "Active": true,
      "FirstDetectedAt": "2026-10-29T09:28:20.770Z"
    },
    {
      "QueueName": "DemoQueue2",
      "PreviousQueuedMessages": 2,
      "QueuedMessages": 2,
      "Active": true,
      "FirstDetectedAt": "2026-10-29T09:28:20.770Z"
    }
  ]
}
```

- El Sender de la integración deberá ser de tipo HTTPS, con un rol que coincida con el configurado en la service key configurada (plan: integration-flow), y con el check de CSRF desmarcado:

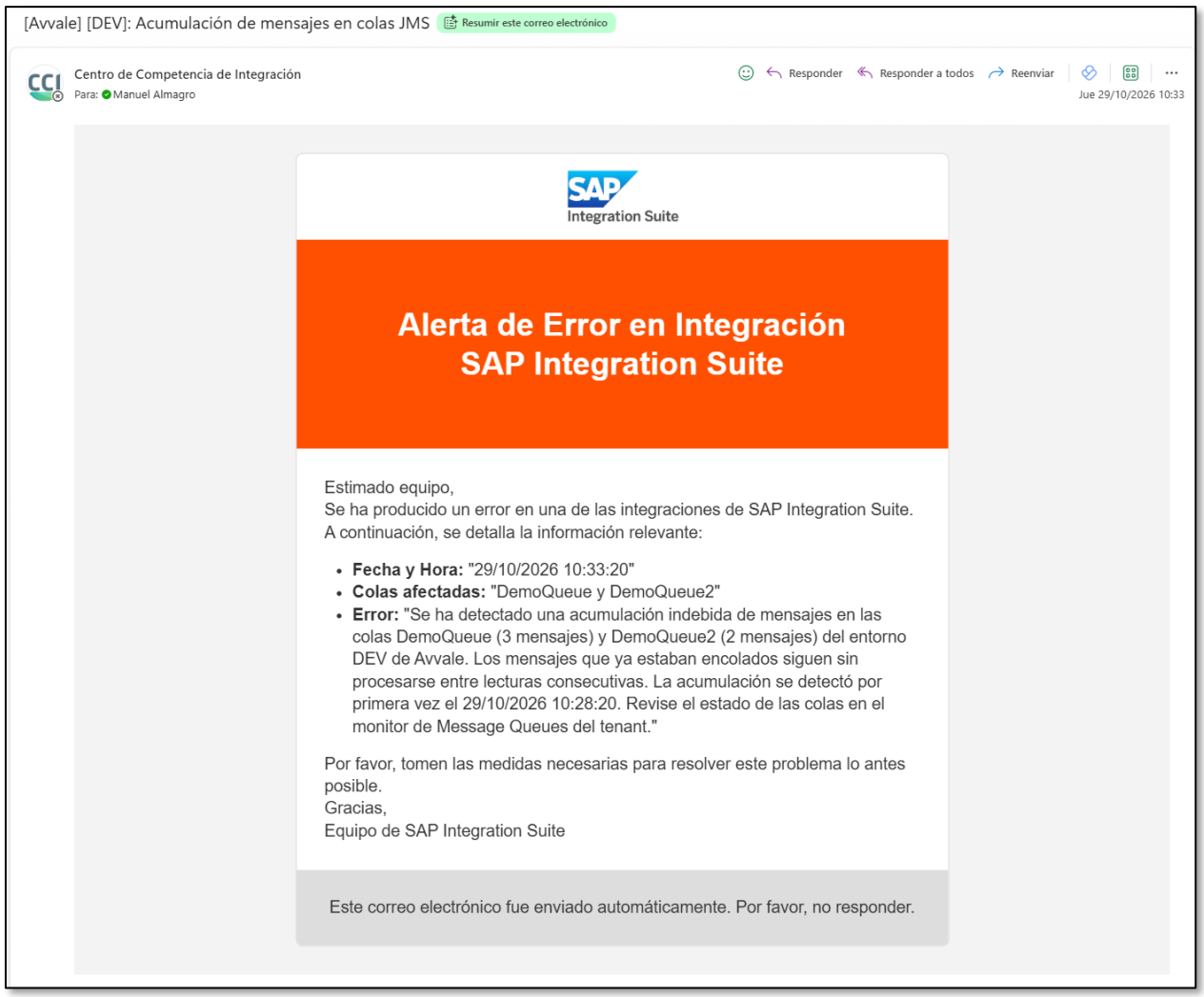


Por lo demás, el equipo de desarrollo de integraciones es libre de hacer el uso que sea necesario de la información que recibe. Se recomienda que la alerta JSON recibida se use para generar un mensaje user-friendly que permita al equipo de monitorización actuar sobre la alerta.

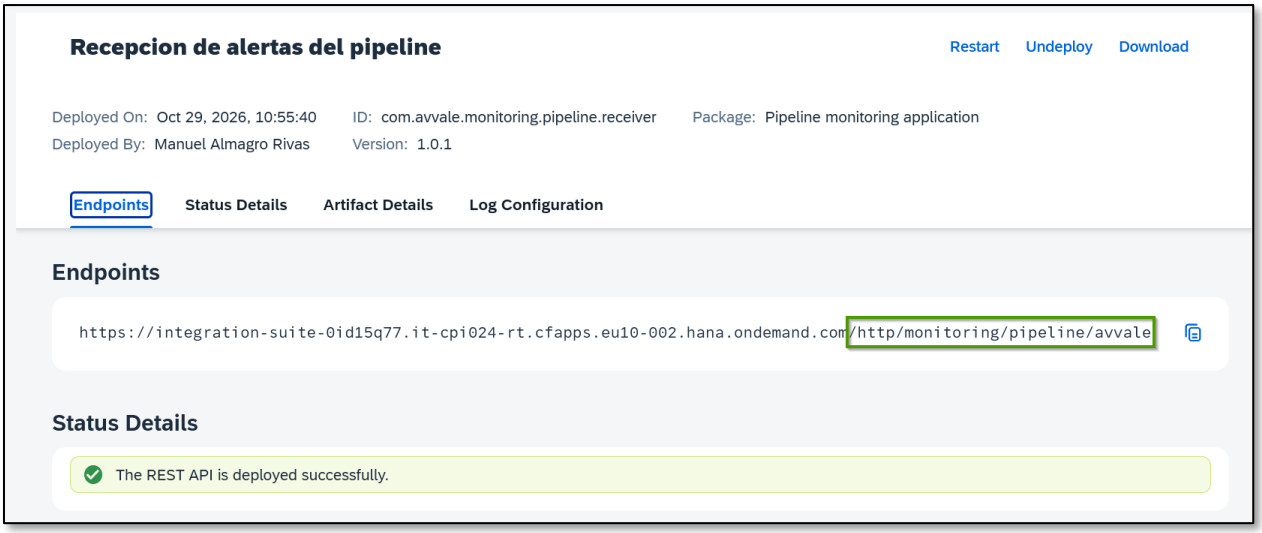
La integración usada en la proof of concept fue la siguiente, a modo de ejemplo:



En el script Groovy se utiliza el JSON para generar el cuerpo de un mail que se envía al equipo de integración, resultando en algo así:



El desarrollador deberá entregar el endpoint al responsable de configurar la app. En particular, deberá entregar el path (pues el host ya aparece en la service key y la aplicación lo extrae de ahí):



Configuración de variables de entorno

Recapitulando, en este punto deberíamos tener lo siguiente:

- Una service key que nos permita consultar la API de producto de Integration Suite (plan: api).
- Una service key que nos permita llamar a una integración que manejará las alertas (plan: integration-flow).
- Una integración configurada según las directrices proporcionadas en este documento, desplegada en el mismo tenant sobre el cual se creó la anterior service key.
- Una copia local del repositorio de la app PipelineMonitoring.
- El archivo vars.yml creado a partir de vars.example.yml, listo para definir variables de entorno.

Es ahora cuando editaremos el archivo vars.yml. Se han de configurar las variables definidas en la plantilla:

1. client_slug: es el slug (el sufijo) que se añadirá al nombre de la app. El objetivo es evitar colisiones en el espacio de nombres, de modo que debe ser algo que identifique el cliente y entorno siendo monitorizados. Por ejemplo: avvale-test; vetclinic-sandbox; scorsese-prod.
2. cf_region: región de Cloud Foundry donde hemos desplegado la app. Puede consultarse en el overview de la subaccount de BTP (véase la sección Despliegue al entorno Cloud Foundry).
3. cpi_api_service_key: aquí pegaremos el JSON de la service key generada para consultar la API de producto (plan: api).

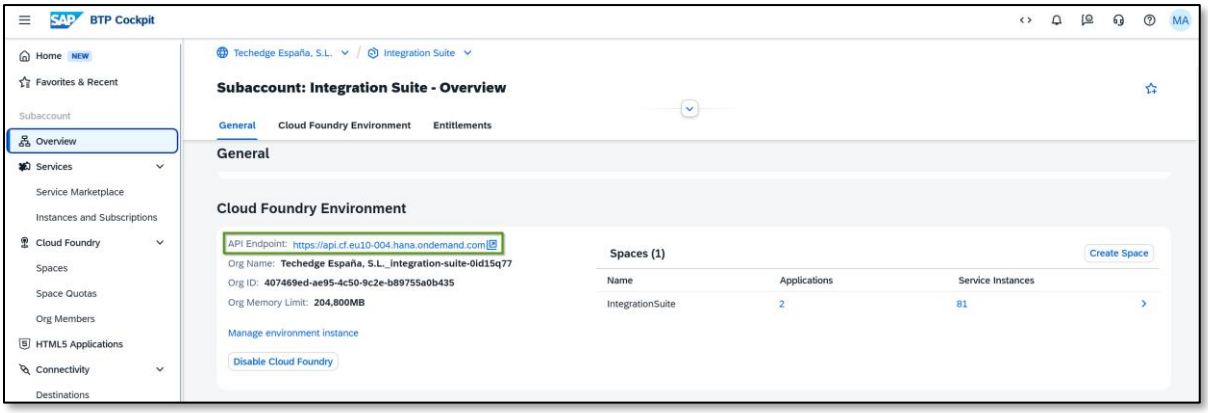
4. alert_service_key: aquí pegaremos el JSON de la service key generada para enviar peticiones al endpoint de integración (plan: integration-suite).
5. alert_endpoint_path: aquí pegaremos el path a la integración que manejará las excepciones (p.e.: /http/monitoring/pipeline/avvale).
6. client_name: aquí definiremos el nombre del cliente tal cual queremos que aparezca en la alerta generada (p.e.: Avvale).
7. environment: aquí definiremos el nombre del entorno tal cual queremos que aparezca en la alerta generada (p.e.: dev).
8. monitored_queues: lista de las colas que queremos monitorizar, separadas por comas, sin espacios; por ejemplo: DemoQueue,DemoQueue2.
9. monitoring_freq_in_minutes: minutos que han de pasar entre comprobaciones. Número entero entre comillas dobles, p.e.: “5”
10. monitoring_wait_in_seconds: segundos que pasarán cada vez que se mira una cola entre la primera y la segunda comprobación. Ha de ser un intervalo sustancialmente menor al definido entre chequeos en el parámetro anterior. Número entero entre comillas dobles, p.e.: “30”

El resto de parámetros son opcionales y su significado se puede consultar en la especificación de la aplicación. Cuando completemos la configuración, guardaremos el archivo.

Despliegue al entorno Cloud Foundry

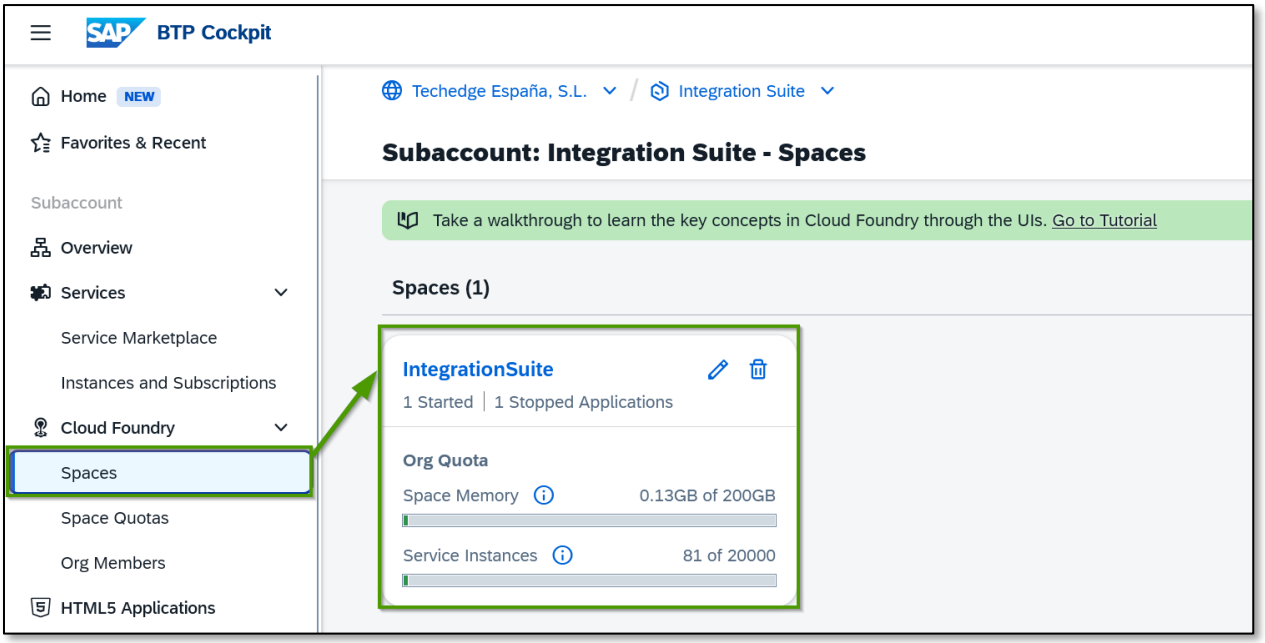
Abriremos una consola (Windows: ⊞ + R y escribir cmd.exe, macOS: ⌘ + espacio y escribir Terminal). Los pasos que seguiremos serán:

- Si no hemos hecho login previamente, escribiremos el siguiente comando que nos llevará por el proceso: `cf login`
- Si se nos solicita una URL, podremos verla yendo a la subaccount de BTP donde se va a desplegar la aplicación > Overview > Cloud Foundry Environment > API Endpoint:

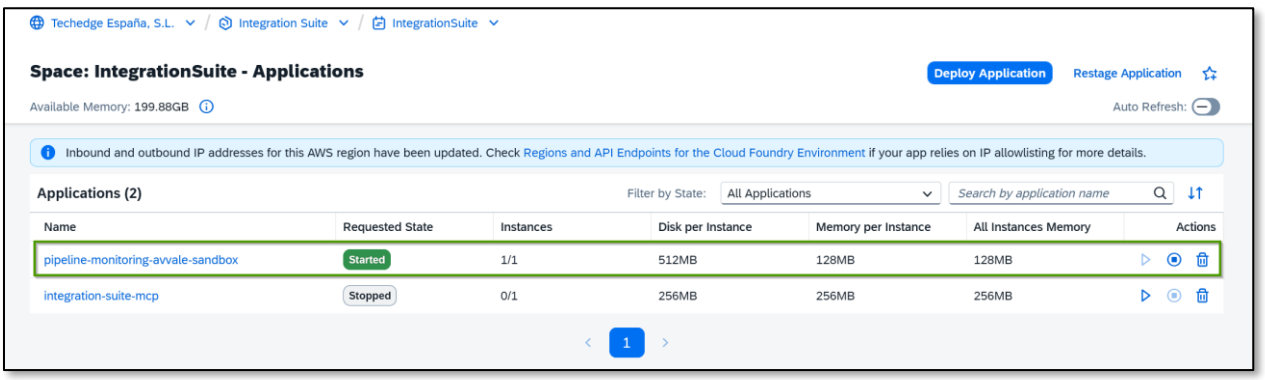


- El propio comando nos pedirá elegir una organización y un espacio. Seleccionaremos aquellos que nos sean relevantes (donde queramos desplegar la aplicación; no tienen por qué coincidir con nada que hayamos hecho antes, ni donde estén las service key).
- Cuando el proceso de login nos devuelva el control de la consola, navegaremos hasta la carpeta donde tenemos la copia del repositorio. Por ejemplo:

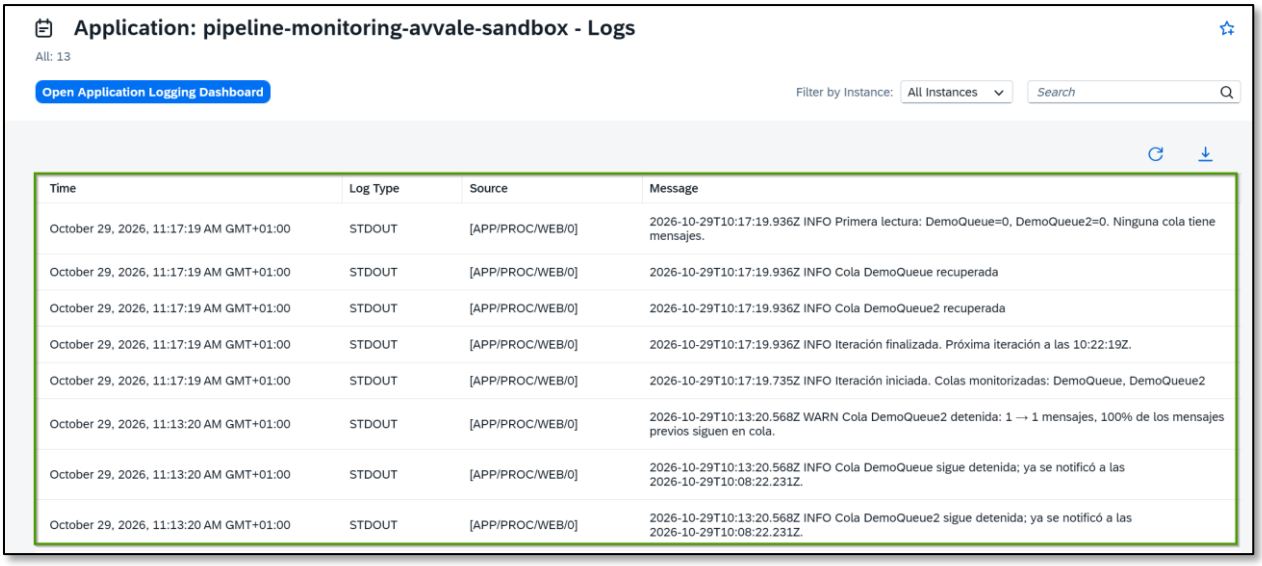
  `cd C:\Users\ManuelAlmagro\Desktop\AvvalePipelineMonitoring`
- Aquí indicaremos que queremos desplegar la aplicación, especificando que nuestras variables de entorno están en el archivo que hemos informado: `cf push --vars-file vars.yml`
- Iremos a la subaccount correspondiente en BTP, Cloud Foundry > seleccionamos el espacio donde la hayamos desplegado:



Aquí veremos nuestra aplicación corriendo:



Haciendo clic en ésta podremos acceder a información útil, como los recursos que consume y (crucialmente) los logs, donde veremos el resultado de cada comprobación:



Con esto concluye la configuración.

de Empresas con la misión de hacer evolucionar los negocios a través de la economía circular habilitada por la tecnología.

avvale

Enabling what's next